Design principles for AI workloads on Azure

This article outlines the core principles for AI workloads on Azure, with a focus on the AI aspects of an architecture. It's essential to consider all Azure Well-Architected Framework pillars collectively, including their trade-offs. Apply each pillar to the functional and nonfunctional requirements of the workload.

Reliability

When you run AI workloads on Azure, you need to consider many of the same reliability requirements that you consider for other types of workloads. However, specific considerations for model training, hosting, and inferencing are particularly important and are the focus of this article. It's important to integrate these practices with standard design best practices for cloud applications, which also apply to AI workloads.

Review the reliability design principles to get a foundational understanding of the concepts described here.

Expand table

Design principle	Considerations
Mitigate single points of failure.	Relying on a single instance for critical components can lead to significant problems. To prevent these kinds of problems, build redundancy into all critical components. Use platforms that have built-in fault tolerance and high-availability features, and implement redundancy by deploying multiple instances or nodes.
Conduct failure mode analysis. Take advantage of well-known design patterns.	Regularly analyze potential failure modes in your system components, and build resilience against these failures. Use well-known design patterns to isolate parts of your system and prevent cascading failures. For example, the Bulkhead pattern can help isolate failures, and retry mechanisms and circuit breakers can handle transient errors like throttling requests.
Balance reliability objectives across dependent components.	Ensure that the inferencing endpoint, or model, and the data store are aligned in terms of reliability. Application components must remain available even under unexpected conditions, such as a surge in concurrent users. Additionally, the workload should be able to fail over to backup systems in case of failures. If one component fails, it affects the reliability of the other. Similarly, service layer APIs, which are critical production resources, should adhere to the same reliability standards as other high-criticality workload flows. If these APIs

Design principle	Considerations
	require high availability, the hosting platform must support availability zones or a multiregion design to ensure continuous operation and resilience.
Design for operational reliability.	Maintain the reliability of model responses by ensuring that updates are frequent and timely. Decide whether to use recent data or slightly older data, balancing the need for up-to-date information with the practicality of update frequency. You should automate online training because of its frequency and reliability requirements. Offline training can be justified through a cost-benefit analysis. You can use cheaper resources for offline training.
Design for a reliable user experience.	Use load testing to evaluate how your workload handles stress, and design your user interface to manage user expectations for response times. Implement UI elements that inform users of potential wait times, and adopt asynchronous cloud design principles to address intermittence, latency, or failures.

Security

Models often use sensitive production data to produce relevant results. You must therefore implement robust security measures in all layers of the architecture. These measures include implementing secure application design early in the lifecycle, encrypting data both at rest and in transit, and adhering to industry compliance standards. Regular security assessments are crucial for identifying and mitigating vulnerabilities. Advanced threat detection mechanisms help ensure prompt responses to potential threats.

Security principles are fundamental for safeguarding data integrity, design integrity, and user privacy in AI solutions. As a workload owner, you're responsible for building trust and protecting sensitive information to help ensure a safe user experience.

Expand table

Design principle	Considerations
Earn user trust.	Integrate content safety at every stage of the lifecycle by using custom code, tools, and effective security controls. Remove unneeded personal and confidential information at all data storage points, including aggregated data stores, search indexes, caches, and applications. Maintain this level of security consistently throughout the architecture. Be sure to implement thorough content moderation that inspects data in both directions and filters out inappropriate and offensive content.

Design principle	Considerations
Protect data at rest, in transit, and in use, according to the sensitivity requirements of the workload.	At a minimum, use platform-level encryption with Microsoft-managed or customer-managed keys on all data stores in the architecture. Determine where you need higher levels of encryption, and use double encryption if you need to. Ensure that all traffic uses HTTPS for encryption. Determine the TLS termination points at each hop. The model itself needs to be protected to prevent attackers from extracting sensitive information that's used during training. The model requires the highest level of security measures. Consider using homomorphic encryption that allows inferencing on encrypted data.
Invest in robust access management for identities (user and system) that access the system.	Implement role-based access control (RBAC) and/or attribute-based access control (ABAC) for both control and data planes. Maintain proper identity segmentation to help protect privacy. Only allow access to content that identities are authorized to view.
Protect the integrity of the design by implementing segmentation.	Provide private networking for access to centralized repositories for container images, data, and code assets. When sharing infrastructure with other workloads creates segmentation, for example, when you host your inference server in Azure Kubernetes Service, isolate the node pool from other APIs and workloads.
Conduct security testing.	Develop a detailed test plan that includes tests for detecting unethical behavior whenever changes are introduced to the system. Integrate AI components into your existing security testing. For instance, incorporate the inferencing endpoint into your routine tests alongside other public endpoints. Consider conducting tests on the live system, such as penetration tests or red team exercises, to identify and address potential vulnerabilities effectively.
Reduce the attack surface by enforcing strict user access and API design.	Require authentication for all inferencing endpoints, including system-to-system calls. Avoid anonymous endpoints. Prefer constrained API design at the cost of client-side flexibility.

 **Trade-off.** Implementing the highest levels of security incurs trade-offs in cost and accuracy because the ability to analyze, inspect, or log the encrypted data is limited. Content safety checks and achieving explainability can also be challenging in highly secured environments.

The following design areas provide details on the preceding principles and considerations:

- Responsible AI
- Testing
- Operations
- MLOps and GenAIOps

Cost Optimization

The goal of the Cost Optimization pillar is to maximize investment, not necessarily to reduce costs. Every architectural choice creates both direct and indirect financial impacts. Understand the costs that are associated with various options, including build versus buy decisions, technology selections, billing models, licensing, training, and operational expenses. It's important to maximize your investment in the selected tier and continuously reassess billing models. Continuously evaluate costs that are associated with changes in architecture, business needs, processes, and team structure.

Review the Cost Optimization principles, focusing on rate and usage optimization.

Expand table

Design principle	Considerations
Determine the cost drivers by doing a comprehensive cost modeling exercise.	Consider the key factors of the data and application platform: - **Volume of data**. Estimate the amount of data you'll be indexing and processing. Larger volumes can increase storage and processing costs. - **Number of queries**. Predict the frequency and complexity of queries. Higher query volumes can increase costs, especially if the queries require significant computational resources. - **Throughput**. Assess the expected throughput to determine which performance tier you need. Higher throughput demands can lead to higher costs. - **Dependency costs**. Understand that there might be hidden costs in dependencies. For example, when you calculate the cost of indexing, include the cost of the skill set, which is part of data processing but outside the index scope.
Pay for what you intend to use.	Choose a technology solution that meets your needs without incurring unnecessary expenses. If you don't need advanced features, consider less expensive options and open-source tools. Factor in the frequency of usage. Prefer elastic compute options for orchestration tools to minimize utilization costs, because they're *always-on*. Avoid serverless compute for full-time operations because it can escalate costs.

Design principle	Considerations
	Don't pay for higher tiers without using their full capacity. Ensure that your chosen tier aligns with your production usage patterns to optimize spending. Use standard pricing for regular tasks and spot pricing for highly interruptible training. To reduce costs, use GPU-based compute only for AI workload functions. Extensively test and benchmark your training and fine-tuning to find the SKU that best balances performance and cost.
Use what you pay for. Minimize waste.	Monitor utilization metrics closely. AI workloads can be expensive, and costs can escalate quickly if resources aren't shut down, scaled down, or deallocated when they're not being used. Optimize the system for *write once, read many* to avoid overspending on data storage. Training data doesn't require the instant responsiveness of a production database. Stress testing an Azure OpenAI Service inference endpoint can be expensive because every call incurs charges. To reduce costs, use unused PTUs of OpenAI Service in a test environment or simulate the inference endpoint instead. Tasks like exploratory data analysis (EDA), model training, and fine-tuning are typically not run full time. For these tasks, prefer a platform that can be stopped when it's not in use. For example, EDA jobs are typically interactive, so users need to be able to start VMs and stop them as they run jobs. Assign cost accountabilities to operations teams. These teams should ensure that costs stay within expected parameters by actively monitoring and managing resource utilization.
Optimize operational costs.	Online training can be expensive because of its frequency requirements. Automating this process helps maintain consistency and minimizes cost from human error. Additionally, using slightly older data for training, and delaying updates when possible, can further reduce expenses without significantly affecting model accuracy. For offline training, evaluate whether cheaper resources can be used, such as offline hardware. In general, delete data from feature stores to reduce clutter and storage costs for features.

Trade-off: Cost Optimization and Performance Efficiency. Balancing cost with performance in database management involves trade-offs. Efficient index design speeds up

> queries but can increase costs because of metadata management and index size. Similarly, partitioning large tables can improve query performance and reduce storage load, but it incurs additional costs. Conversely, techniques that avoid excessive indexing can reduce costs but might affect performance if they're not managed properly.

Operational Excellence

The primary objective of Operational Excellence is to deliver capabilities efficiently throughout the development lifecycle. Achieving this goal involves establishing repeatable processes that support the design methodology of experimentation and yield results to improve model performance. Operational Excellence is also about maintaining the accuracy of models over time, implementing effective monitoring practices and governance to minimize risks, and developing change management processes to adapt to model drift.

Although all Operational Excellence principles apply to AI workloads, prioritize automation and monitoring as your foundational operational strategies.

Expand table

Design principle	Considerations
Foster a continuous learning and experimentation mindset throughout the lifecycles of application development, data handling, and AI model management.	Workload operations should build on industry-proven methodologies like DevOps, DataOps, MLOps, and GenAIOps. Early collaboration between operations, application development, and data teams is essential to establish a mutual understanding of acceptable model performance. Operations teams provide quality signals and actionable alerts. Application and data teams help diagnose and resolve problems efficiently.
Choose technologies that minimize operational burden.	When you choose platform solutions, prefer platform as a service (PaaS) over self-hosted options to simplify design, automate workflow orchestration, and make day-2 operations easier.
Create an automated monitoring system that supports alerting functionality, including logging and auditability.	Given the nondeterministic nature of AI, it's important to establish quality measurements early in the lifecycle. Work with data scientists to define quality metrics. Visualize ongoing insights in comprehensive dashboards. Track experiments by using tools that can capture details like code versions, environments, parameters, runs, and results. Implement actionable alerts that provide just enough information to enable operators to respond quickly.

Design principle	Considerations
Automate the detection and mitigation of model decay.	Use automated tests to evaluate drift over time. Ensure that your monitoring system sends alerts when responses start deviating from the expected results and start doing so on a regular basis. Use tools that can automatically discover and update new models. As needed, adapt to new use cases by adjusting data processing and model training logic.
Implement safe deployments.	Choose between side-by-side deployments or in-place updates to minimize downtime. Implement thorough testing before deployment to ensure the model is correctly configured and meets targets, user expectations, and quality standards. Always plan for emergency operations, regardless of the deployment strategy.
Continuously evaluate the user experience in production.	Enable workload users to provide feedback about their experience. Get consent to share part or all of their conversations in an associated log for troubleshooting. Consider how much of a user interaction is viable, compliant, safe, and useful to capture, and use the data diligently to evaluate the performance of your workload with real user interactions.

The following design areas provide details on the preceding principles and considerations:

- Operations
- MLOps and GenAIOps
- Testing

Performance Efficiency

The goal of AI model performance evaluation is to determine how effectively a model accomplishes its intended tasks. Accomplishing this goal involves assessing various metrics, like accuracy, precision, and fairness. Additionally, the performance of the platform and application components that support the model is crucial.

Model performance is also influenced by the efficiency of operations like experiment tracking and data processing. Applying Performance Efficiency principles helps measure performance against an acceptable quality bar. This comparison is key to detecting degradation or decay. To maintain the workload, including AI components, you need automated processes for continuous monitoring and evaluation.

Expand table

Design principle	Considerations
Establish performance benchmarks.	Conduct rigorous performance testing across different architectural areas, and set acceptable targets. This ongoing assessment should be part of your operational processes, not a one-time test. Benchmarking applies to prediction performance. Start with a baseline to understand initial model performance and continuously re-evaluate performance to ensure that it meets expectations.
Evaluate resource needs for meeting performance targets.	Understand load characteristics to choose the right platform and right-size resources. Factor in this data for capacity planning to operate at scale. For example, use load testing to determine the optimal compute platform and SKU. High-performance GPU-optimized compute is often needed for model training and fine-tuning, but general-purpose SKUs are suitable for orchestration tools. Similarly, choose a data platform that efficiently handles data ingestion, manages concurrent writes, and maintains individual write performance without degradation. Different inferencing servers have varying performance characteristics. These characteristics affect model performance at runtime. Select a server that meets baseline expectations.
Collect and analyze performance metrics, and identify bottlenecks.	Evaluate telemetry from the data pipeline to ensure that performance targets for throughput and read/write operations are met. For the search index, consider metrics like query latency, throughput, and the accuracy of the results. As data volume increases, error rates shouldn't increase. Analyze telemetry from code components, like the orchestrator, which collects data from service calls. Analyzing that data can help you understand time spent on local processing versus network calls and identify potential latency in other components. Assess the UI experience by using engagement metrics to determine whether users are positively engaged or frustrated. Increased time on a page can indicate either. Multimodal capabilities, like voice or video responses, can cause significant latency, leading to longer response times.
Continuously improve benchmark performance by using quality measurements from production.	Implement automated collection and analysis of performance metrics, alerts, and model retraining to maintain model efficacy.

> **Trade-off**. When you consider platform capabilities, you need to balance cost and performance. For example, GPU SKUs are expensive, so continuously monitor for under-utilization and right-size resources as needed. After adjustments, test resource utilization to ensure a balance between the cost of platform resources and their operations and the expected performance, as indicated by the baseline. For Cost Optimization strategies, see Recommendations for optimizing component costs.

The following design areas provide details on the preceding principles and considerations:

- Testing
- MLOps and GenAIOps

Next step

Design area: Application design

Last updated on 01/09/2025